
Лабораторная работа: скалярное автоматическое дифференцирование и обучение одного нейрона

Содержание

Как пользоваться этой работой	1
Учебный контракт	1
Подготовка пустого notebook	1
Часть первая. Скалярное автоматическое дифференцирование	2
Создаём каркас Value	2
Как Python представляет вычислительный граф	2
Почему локальная функция backward записывается заранее	3
Как выражение Python становится вершиной графа	5
Добавляем сложение	5
Добавляем умножение	6
Добавляем отрицание, вычитание и степень	7
Добавляем элементарные функции	8
Строим обратный проход	9
Как Python выполняет backward	9
Ручная трассировка трёх локальных шагов	10
Полный итоговый текст ячейки Value	11
Проверяем градиент центральной разностью	13
Часть вторая. Обучаем один нейрон	14
Математическая модель	14
Создаём класс TanhNeuron	14
Объекты параметров и граф эпохи	17
Строим среднеквадратичную ошибку	17
Обучаем параметры	18
Диагностика	19
Ошибка: у общего узла неверный градиент	19
Ошибка: все градиенты равны нулю	19
Ошибка: появляется неверный порядок вычислений	20
Ошибка: loss не уменьшается	20
Ошибка: после второго backward градиент неожиданно удваивается	20
Что здесь намеренно не разбирается	20
Чек-лист перед сдачей	20
Вопросы для самопроверки	20

Как пользоваться этой работой

Эта работа выполняется в Jupyter notebook без библиотек машинного обучения. Откройте файл `student_autograd_lab.ipynb` и читайте этот guide сверху вниз. В notebook уже есть Markdown-ячейки с названиями этапов и пустые code-ячейки с идентификаторами. Переносите код только из этого guide в ячейку с тем же идентификатором.

Не копируйте код из `teacher_autograd_lab_solution.ipynb` до самостоятельной попытки. В этом guide приведён весь код, который будет в образце преподавателя: ни одна строка решения не скрыта в другом файле.

Работа рассчитана на расширенный маршрут продолжительностью примерно от двух до трёх часов. Теория не является необязательным приложением: сначала прочитайте [theory.md](#), затем переносите код и выполняйте эксперименты.

Работа состоит из двух частей.

1. Построить скалярный механизм автоматического дифференцирования Value.
2. Обучить один нейрон с помощью созданного механизма.

Выполняйте действия в порядке guide. После изменения класса всегда повторно запускайте всю его ячейку, а затем эксперимент текущего этапа. Старые объекты Value были созданы старой версией класса и не должны смешиваться с новой версией.

Учебный контракт

После завершения работы вы должны уметь:

- объяснить разницу между числом data и градиентом grad;
- нарисовать вычислительный граф маленького выражения;
- применить правило цепочки к сложению, умножению и гиперболическому тангенсу;
- объяснить, почему градиенты ветвящегося графа накапливаются;
- объяснить, зачем нужен обратный топологический порядок;
- проверить градиент центральной разностью;
- обучить параметры w и b одного нейрона по функции потерь.

Перед началом полностью прочитайте [теорию](#). В guide теория повторяется ровно в тот момент, когда она нужна для очередного действия.

Подготовка пустого notebook

Создайте или откройте `student_autograd_lab.ipynb`. Не устанавливайте пакеты. Достаточно стандартной библиотеки Python. В первой code-ячейке с идентификатором `imports_and_helper` вставьте полный текст.

```
# AUTOGRAD_ESSENTIALS_CELL: imports_and_helper
import math

def assert_close(actual, expected, tolerance=1e-9):
    difference = abs(actual - expected)
    assert difference <= tolerance, (
        f"Ожидалось {expected}, получено {actual}, "
        f"абсолютная ошибка {difference}"
    )
```

Функция `assert_close` сравнивает вещественные числа с допустимой погрешностью. Строгое сравнение через `==` для результатов тригонометрических и экспоненциальных вычислений часто ненадёжно из-за представления вещественных чисел в памяти.

Запустите ячейку. Она не должна печатать текст и не должна выдавать ошибку.

Часть первая. Скалярное автоматическое дифференцирование

Создаём каркас Value

Каждая вершина графа будет экземпляром Value. Сначала вставьте в ячейку value начальный каркас. Этот каркас нужен только как место для последовательного заполнения методов. Не запускайте эксперименты, которым требуется ещё не реализованная операция.

```
class Value:
    def __init__(self, data, parents=(), op=""):
        self.data = float(data)
        self.grad = 0.0
        self.parents = parents
        self.op = op
        self._backward = lambda: None

    def __repr__(self):
        return (
            f"Value(data={self.data:.6g}, grad={self.grad:.6g}, "
            f"op={self.op!r})"
        )
```

Значение data всегда приводится к float. Это убирает различие между целым числом 2 и вещественным числом 2.0 для нашей учебной реализации. Поле grad сначала равно нулю: до вызова backward мы ещё не нашли влияние итоговой функции на эту вершину.

Проверьте каркас вручную:

```
first_value = Value(2)
assert first_value.data == 2.0
assert first_value.grad == 0.0
assert first_value.parents == ()
assert first_value.op == ""
print(first_value)
```

Ожидается объект с data, равным 2, и grad, равным 0.

Как Python представляет вычислительный граф

В этой работе граф не хранится в отдельной специальной структуре данных. Его вершина — обычный изменяемый объект Python класса Value. Ссылка на этот объект, например переменная a, обозначает именно одну вершину, а не только её число a.data.

Поле parents хранит кортеж ссылок на непосредственных родителей вершины. Например, после result = left + right новый объект result хранит parents=(left, right). Значения данных на прямом проходе движутся от left и right к result, однако в памяти мы сохраняем ссылки в обратную сторону — от результата к его родителям. Поэтому, начиная с итоговой вершины, метод backward может добраться до всего нужного подграфа.

Каждая реализованная операция создаёт новый объект результата, который ссылается только на уже существующие объекты. В рамках одного прямого прохода это исключает цикл: новое значение не может стать родителем самого себя. Так Python постепенно строит ориентированный ациклический граф во время выполнения обычных выражений.

Одна и та же ссылка может встретиться в parents несколько раз. В выражении a * a + a объект a не копируется: у результата a * a будет кортеж parents=(a, a), а затем этот же объект a станет вторым родителем сложения. Это несколько рёбер, ведущих к одной вершине, и именно поэтому её градиент должен накопить несколько вкладов.

Поля вершины имеют разные роли:

- data хранит число, вычисленное на прямом проходе;
- grad — изменяемое поле, в которое обратный проход складывает вклады;

- `parents` задаёт ссылки на непосредственные входы операции;
- `op` хранит только метку операции для чтения и диагностики, метод `backward` не использует её для вычисления;
- `_backward` хранит локальное правило обратного прохода этой вершины.

Почему локальная функция `backward` записывается заранее

До реализации первой операции важно увидеть связь между формулой и будущей строкой кода. Пусть текущая операция вычислила $z = f(x, y)$, а итоговая скалярная функция всего графа равна J . Если во время обратного прохода уже известен градиент результата операции

$$g_z = \frac{\partial J}{\partial z},$$

то правило цепочки даёт вклад текущей операции в градиент родителя x :

$$\Delta g_x^{(z)} = g_z \frac{\partial z}{\partial x}.$$

Аналогично для y :

$$\Delta g_y^{(z)} = g_z \frac{\partial z}{\partial y}.$$

Поле `result.grad` хранит g_z . Поля `self.grad` и `other.grad` накапливают полные градиенты родителей. Поэтому в коде всегда используется `+=`:

$$g_x \leftarrow g_x + \Delta g_x^{(z)}.$$

Сейчас, в прямом проходе, `result.grad` ещё равен нулю. Мы не вычисляем градиент заранее. Мы только создаём и сохраняем вложенную функцию `backward` вместе с локальной формулой. Это вложенная функция, или замыкание Python: она сохраняет ссылки на объекты `self`, `other` и `result`, доступные в момент её создания. Строка `result._backward = backward` помещает сам объект функции в новую вершину графа.

Позже метод `Value.backward` начнёт от итоговой вершины, у которой `grad` равен 1, и будет идти против направления рёбер. Когда затем вызывается `result._backward()`, замыкание читает текущее, уже накопленное `result.grad`, а также сохранённые `self.data` и `other.data`. Оно не хранит заранее вычисленный градиент и не запускается при создании `result`. Каждый потомок `result` сначала прибавит в `result.grad` свой вклад. Только когда все потомки уже обработаны, будет вызвана сохранённая функция `result._backward`. Поэтому к этому моменту `result.grad` содержит полный g_z , а не случайное или неполное число. Метод `build` и `reversed(topo)`, которые вы реализуете ниже, обеспечивают этот порядок для всего графа.

Из этого следует важное практическое правило: после построения `loss` нельзя менять `data` параметров до вызова `loss.backward()`. Например, локальное правило умножения читает `other.data` во время обратного прохода. Если изменить параметр раньше, замыкание использует новое число вместо числа того прямого прохода, для которого строился граф. В эпохе обучения сначала выполняется `loss.backward()`, а обновление `parameter.data` происходит только после него.

У листовой вершины нет локальной операции и родителей. Для неё начальное значение `_backward = lambda: None` — пустая функция без действия. Благодаря этому цикл обратного прохода может одинаково вызвать `_backward()` у любой вершины, включая лист.

Читайте рисунок по номерам: итоговая функция сначала передаёт градиент своим потомкам, потомки складывают вклады в общий узел `result`, и только после этого `result` использует накопленное значение для передачи градиента своим родителям.

Почему g_z известен до обработки узла z

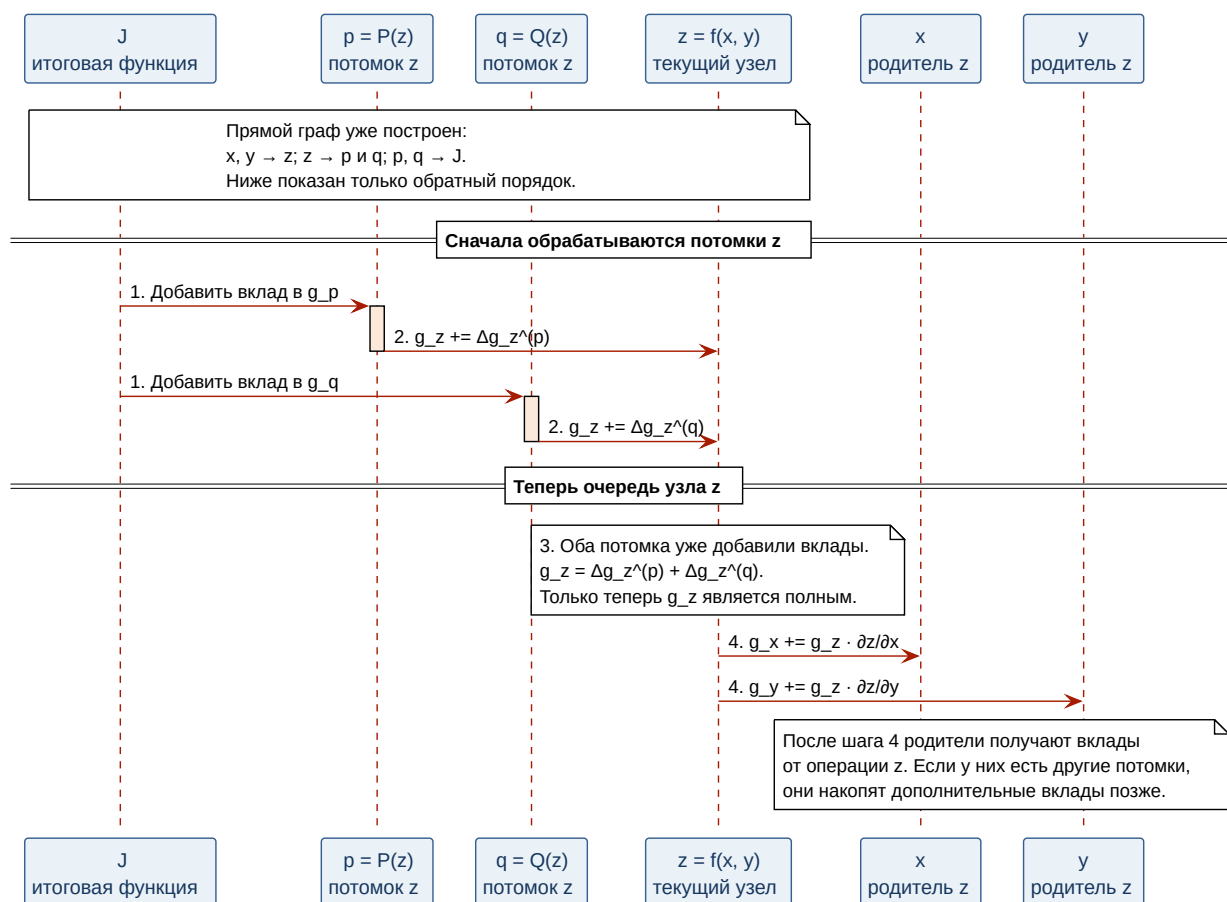


Рис. 1. Почему grad результата уже полон

Как выражение Python становится вершиной графа

Методы с двойными подчёркиваниями — специальные методы Python, которые позволяют объекту участвовать в обычной арифметической записи. Когда интерпретатор видит `left + right` и `left` является `Value`, он вызывает `left.__add__(right)`. Аналогично `left * right` вызывает `__mul__`, а унарная запись `-value` вызывает `__neg__`. Каждый из этих методов возвращает новый `Value`, поэтому для студента выражение выглядит как арифметика, а для реализации одновременно создаётся новая вершина графа.

Если `Value` находится справа от обычного числа, например в `3 + value` или `3 * value`, Python сначала пытается выполнить операцию числа. Когда она не подходит для `Value`, вызываются отражённые методы `value.__radd__(3)` и `value.__rmul__(3)`. Поэтому строки `__radd__ = __add__` и `__rmul__ = __mul__` позволяют одной реализации обслуживать оба порядка операндов.

Проверка `isinstance(other, Value)` отделяет уже существующую вершину от обычного числа. Для числа создаётся новый константный лист `Value(other)`. Его `grad` при необходимости тоже может получить значение, но метод `parameters()` нейрона не возвращает такие временные константы, поэтому градиентный спуск их не обновляет. Вычитание не строит новую особую операцию графа: `x - y` раскрывается как `x + (-y)` и использует уже созданные вершины отрицания и сложения.

Добавляем сложение

Для $z = x + y$ локальные производные равны единице:

$$\frac{\partial z}{\partial x} = 1,$$

$$\frac{\partial z}{\partial y} = 1.$$

Внутри тела класса `Value` сразу после **repr** добавьте этот метод и строку присваивания. Если `other` является обычным числом, его надо превратить в `Value`. Благодаря этому выражения вида `value + 3` будут работать так же, как `value + Value(3)`.

```
def __add__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    result = Value(
        self.data + other.data,
        parents=(self, other),
        op="+",
    )

    def backward():
        self.grad += result.grad
        other.grad += result.grad

    result._backward = backward
    return result

__radd__ = __add__
```

Строка **radd = add** нужна для выражения `3 + value`. В этом случае Python сначала попытает специальный метод числа, а затем вызовет отражённое сложение у `Value`.

Когда `backward` будет выполнен, для сложения локальные множители равны единице. Поэтому `result.grad` просто прибавляется обоим родителям. Это не означает, что у родителей всегда будет одинаковый итоговый `grad`: в другие операции они могут получить дополнительные вклады.

После вставки метода повторно запустите ячейку `value` и перенесите в ячейку `addition_experiment` полный эксперимент:

```
# AUTOGRAD_ESSENTIALS_CELL: addition_experiment
left = Value(2.5)
right = Value(-1.0)
sum_value = left + right

assert_close(sum_value.data, 1.5)
assert sum_value.parents == (left, right)
assert sum_value.op == "+"
print("Сложение в прямом проходе: OK")
```

Пока мы проверили только прямой проход. Градиенты появятся после реализации метода backward всего графа.

Добавляем умножение

Для $z = xy$ локальные производные зависят от второго множителя:

$$\frac{\partial z}{\partial x} = y,$$

$$\frac{\partial z}{\partial y} = x.$$

Добавьте в тело класса Value следующий метод и строку отражённого умножения:

```
def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    result = Value(
        self.data * other.data,
        parents=(self, other),
        op="*",
    )

    def backward():
        self.grad += result.grad * other.data
        other.grad += result.grad * self.data

    result._backward = backward
    return result

__rmul__ = __mul__
```

Здесь очень важно, что для градиента по self используется other.data, а для градиента по other используется self.data. Это следует прямо из локальных производных произведения. Когда потомки уже накопили result.grad, эти строки реализуют $\Delta g_x = g_z y$ и $\Delta g_y = g_z x$.

Повторно запустите ячейку value, затем вставьте эксперимент:

```
# AUTOGRAD_ESSENTIALS_CELL: multiplication_experiment
left = Value(4.0)
right = Value(-0.5)
product_value = left * right
scaled_value = 3.0 * left

assert_close(product_value.data, -2.0)
assert_close(scaled_value.data, 12.0)
assert product_value.parents == (left, right)
assert product_value.op == "*"
print("Умножение в прямом проходе: OK")
```

Добавляем отрицание, вычитание и степень

Отрицание использует формулу $z = -x$ и локальную производную -1 . Вычитание раскрывается как $x - y = x + (-y)$. Для степени с постоянным показателем p используется формула:

$$\frac{d}{dx}x^p = px^{p-1}.$$

Добавьте в тело класса Value все методы из этого блока:

```
def __neg__(self):
    result = Value(-self.data, parents=(self,), op="neg")

    def backward():
        self.grad += result.grad * (-1)

    result._backward = backward
    return result

def __sub__(self, other):
    return self + (-other)

def __rsub__(self, other):
    return other + (-self)

def __pow__(self, power):
    result = Value(
        self.data ** power,
        parents=(self,),
        op=f"pow({power})",
    )

    def backward():
        local_derivative = power * self.data ** (power - 1)
        self.grad += result.grad * local_derivative

    result._backward = backward
    return result
```

Показатель power — обычное число. Наша учебная версия не строит производную по показателю.

Повторно запустите ячейку value и перенесите эксперимент:

```
# AUTOGRAD_ESSENTIALS_CELL: neg_sub_power_experiment
source = Value(3.0)
negative = -source
difference = 5.0 - source
cube = source ** 3

assert_close(negative.data, -3.0)
assert_close(difference.data, 2.0)
assert_close(cube.data, 27.0)
assert negative.op == "neg"
assert cube.op == "pow(3)"
print("Отрицание, вычитание и степень в прямом проходе: OK")
```

Добавляем элементарные функции

Добавьте методы синуса, экспоненты, натурального логарифма и гиперболического тангенса. Их локальные производные:

$$\frac{d}{dx} \sin(x) = \cos(x),$$

$$\frac{d}{dx} \exp(x) = \exp(x),$$

$$\frac{d}{dx} \log(x) = \frac{1}{x},$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x).$$

Добавьте в тело класса Value следующий блок:

```
def sin(self):
    result = Value(math.sin(self.data), parents=(self,), op="sin")

    def backward():
        self.grad += result.grad * math.cos(self.data)

    result._backward = backward
    return result

def exp(self):
    result = Value(math.exp(self.data), parents=(self,), op="exp")

    def backward():
        self.grad += result.grad * result.data

    result._backward = backward
    return result

def log(self):
    result = Value(math.log(self.data), parents=(self,), op="log")

    def backward():
        self.grad += result.grad * (1 / self.data)

    result._backward = backward
    return result

def tanh(self):
    result = Value(math.tanh(self.data), parents=(self,), op="tanh")

    def backward():
        self.grad += result.grad * (1 - result.data ** 2)

    result._backward = backward
    return result
```

Для log обязательно соблюдайте условие $x > 0$. Иначе математическая функция не определена в вещественных числах и Python выдаст ошибку.

Повторно запустите ячейку value и перенесите эксперимент:

```
# AUTOGRAD_ESSENTIALS_CELL: elementary_experiment
angle = Value(0.0)
positive = Value(2.0)
```

```
activation_input = Value(0.7)

assert_close(angle.sin().data, 0.0)
assert_close(positive.exp().log().data, 2.0)
assert_close(activation_input.tanh().data, math.tanh(0.7))
print("Элементарные функции в прямом проходе: OK")
```

Строим обратный проход

Нужно обойти граф так, чтобы итоговая вершина обработалась первой, а исходные вершины — последними. Это не новый граф: метод `backward` обходит тот же сохранённый ориентированный ациклический граф, но против направления данных. Рекурсивная функция `build` добавляет вершину после всех её родителей. Поэтому список `topo` содержит родителей раньше потомков. Затем `reversed(topo)` идёт от результата к исходным данным.

Перед обратным проходом обнулите `grad` у всех вершин текущего графа. Потом положите в `grad` итоговой вершины единицу:

$$\frac{\partial J}{\partial J} = 1.$$

Добавьте в тело класса `Value` метод:

```
def backward(self):
    topo = []
    visited = set()

    def build(node):
        if node not in visited:
            visited.add(node)
            for parent in node.parents:
                build(parent)
            topo.append(node)

    build(self)

    for node in topo:
        node.grad = 0.0

    self.grad = 1.0

    for node in reversed(topo):
        node._backward()
```

Как Python выполняет `backward`

Список `topo` и множество `visited` — обычные контейнеры Python. У рекурсивной функции `build` есть точный инвариант: сразу после выполнения `topo.append(node)` все родители `node`, а также их родители, уже добавлены в `topo`. Это верно потому, что цикл `for parent in node.parents` полностью завершает рекурсивные вызовы до добавления самой вершины. Следовательно, `reversed(topo)` вызывает `_backward()` у потомка раньше, чем у его родителя.

В текущем классе `Value` не определён метод `__eq__`. Поэтому `set visited` различает объекты по их идентичности, а не по совпадению чисел в `data`: `Value(2.0)` и другой `Value(2.0)` — две разные вершины, а две ссылки на один объект `a` — одна вершина. Это ровно то свойство, которое нужно графу.

Для `a * a` кортеж родителей содержит два ребра `(a, a)`. `visited` не даёт добавить вершину `a` в `topo` дважды, однако он не удаляет ни одно ребро и не меняет локальную функцию умножения. Когда вызывается сохранённое `_backward`, его две строки по-прежнему прибавляют два вклада в `a.grad`. Значит, `visited` устраняет только повторный обход вершины, а оператор `+=` сохраняет оба математически необходимых вклада.

Пусть в подграфе, достижимом из итоговой вершины, V вершин и E ссылок родительского типа. `build` добавляет каждую вершину в `visited` и `topo` один раз, а цикл по `parents`

рассматривает каждую ссылку при обработке её потомка. Поэтому построение порядка и обратный проход требуют $O(V + E)$ времени. Список `topo`, множество `visited` и глубина рекурсии в худшем случае используют $O(V)$ дополнительной памяти. Для учебных маленьких графов рекурсивная запись удобна; в промышленной реализации очень длинные цепочки часто обходят без рекурсии.

Повторно запустите ячейку `value`. Затем вставьте эксперимент с общим узлом. Он проверяет, что градиент по `a` складывает вклад от ветви `a * a` и отдельный вклад от `+ a`:

$$f(a) = a^2 + a,$$

$$\frac{df}{da} = 2a + 1.$$

При $a = 2$ ожидается градиент 5.

```
# AUTOGRAD_ESSENTIALS_CELL: backward_shared_graph
a = Value(2.0)
objective = a * a + a
objective.backward()

assert_close(objective.data, 6.0)
assert_close(a.grad, 5.0)
assert_close(objective.grad, 1.0)
print("Обратный проход и накопление двух вкладов: ОК")
```

Если здесь получается 1 или 4 вместо 5, проверьте, что локальные правила используют `+=`, а не `=`.

Ручная трассировка трёх локальных шагов

Метод `backward` обычно скрывает отдельные вызовы локальных функций. В этом учебном опыте мы намеренно вызываем их вручную в правильном обратном порядке. Это не дополнительный механизм и не иной вид графа: те же функции `_backward` будут позднее вызваны циклом `for node in reversed(topo)`.

Возьмём граф из теории:

$$m = ab,$$

$$s = m + a,$$

$$J = \tanh(s),$$

где $a = 0.5$ и $b = 1$. После того как J получает начальный градиент 1, выполните его локальное правило. Затем выполните локальное правило s , затем m . После каждого шага смотрите, какая вершина впервые получила `grad` и какая получила дополнительный вклад.

```
# AUTOGRAD_ESSENTIALS_CELL: manual_backward_trace
a = Value(0.5)
b = Value(1.0)
m = a * b
s = m + a
j = s.tanh()

for node in (a, b, m, s, j):
    node.grad = 0.0

j.grad = 1.0
expected_gs = 1.0 - j.data ** 2
```

```

j._backward()
assert_close(s.grad, expected_gs)
assert_close(m.grad, 0.0)
assert_close(a.grad, 0.0)
print("После J._backward():", "g_s =", s.grad)

s._backward()
assert_close(m.grad, expected_gs)
assert_close(a.grad, expected_gs)
print("После s._backward():", "g_m =", m.grad, "; первый вклад в g_a =", a.grad)

m._backward()
assert_close(a.grad, expected_gs * (1.0 + b.data))
assert_close(b.grad, expected_gs * a.data)
print("После m._backward():", "g_a =", a.grad, "; g_b =", b.grad)

```

В реальной реализации не вызывайте `_backward` вручную во время обучения. Здесь это сделано только для наблюдения за тремя локальными правилами. Обычный вызов `j.backward` самостоятельно построит торо, обнулит градиенты, задаст `j.grad = 1` и вызовет те же функции в той же последовательности.

Полный итоговый текст ячейки Value

Теперь ваша ячейка value должна полностью совпадать с этим текстом. Это тот же полный код, который будет использовать notebook преподавателя.

```

# AUTOGRAD_ESSENTIALS_CELL: value
class Value:
    def __init__(self, data, parents=(), op=""):
        self.data = float(data)
        self.grad = 0.0
        self.parents = parents
        self.op = op
        self._backward = lambda: None

    def __repr__(self):
        return (
            f"Value(data={self.data:.6g}, grad={self.grad:.6g}, "
            f"op={self.op!r})"
        )

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        result = Value(
            self.data + other.data,
            parents=(self, other),
            op="+",
        )

        def backward():
            self.grad += result.grad
            other.grad += result.grad

        result._backward = backward
        return result

    __radd__ = __add__

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)

```

```

    result = Value(
        self.data * other.data,
        parents=(self, other),
        op="*",
    )

    def backward():
        self.grad += result.grad * other.data
        other.grad += result.grad * self.data

    result._backward = backward
    return result

__rmul__ = __mul__

def __neg__(self):
    result = Value(-self.data, parents=(self,), op="neg")

    def backward():
        self.grad += result.grad * (-1)

    result._backward = backward
    return result

def __sub__(self, other):
    return self + (-other)

def __rsub__(self, other):
    return other + (-self)

def __pow__(self, power):
    result = Value(
        self.data ** power,
        parents=(self,),
        op=f"pow({power})",
    )

    def backward():
        local_derivative = power * self.data ** (power - 1)
        self.grad += result.grad * local_derivative

    result._backward = backward
    return result

def sin(self):
    result = Value(math.sin(self.data), parents=(self,), op="sin")

    def backward():
        self.grad += result.grad * math.cos(self.data)

    result._backward = backward
    return result

def exp(self):
    result = Value(math.exp(self.data), parents=(self,), op="exp")

    def backward():
        self.grad += result.grad * result.data

    result._backward = backward
    return result

```

```

def log(self):
    result = Value(math.log(self.data), parents=(self,), op="log")

    def backward():
        self.grad += result.grad * (1 / self.data)

    result._backward = backward
    return result

def tanh(self):
    result = Value(math.tanh(self.data), parents=(self,), op="tanh")

    def backward():
        self.grad += result.grad * (1 - result.data ** 2)

    result._backward = backward
    return result

def backward(self):
    topo = []
    visited = set()

    def build(node):
        if node not in visited:
            visited.add(node)
            for parent in node.parents:
                build(parent)
            topo.append(node)

    build(self)

    for node in topo:
        node.grad = 0.0

    self.grad = 1.0

    for node in reversed(topo):
        node._backward()

```

Проверяем градиент центральной разностью

Построим выражение:

$$f(x) = \sin(x) \exp(x) + x^3.$$

Аналитический градиент найдёт Value. Численный градиент вычислим по формуле:

$$\frac{f(x+h) - f(x-h)}{2h}.$$

Вставьте следующий полный код в ячейку finite_difference:

```

# AUTOGRAD_ESSENTIALS_CELL: finite_difference
def plain_function(x):
    return math.sin(x) * math.exp(x) + x ** 3

point = 0.37
step = 1e-6

input_value = Value(point)

```

```

objective = input_value.sin() * input_value.exp() + input_value ** 3
objective.backward()

numerical_gradient = (
    plain_function(point + step) - plain_function(point - step)
) / (2 * step)
absolute_error = abs(input_value.grad - numerical_gradient)

assert absolute_error < 1e-5, absolute_error
print("Градиент Value      :", input_value.grad)
print("Градиент разностью  :", numerical_gradient)
print("Абсолютная ошибка   :", absolute_error)

```

Если ошибка велика, не увеличивайте допустимую ошибку автоматически. Сначала проверьте локальное правило операции, построение toro и обратный порядок обхода.

Часть вторая. Обучаем один нейрон

Математическая модель

Один нейрон принимает x , хранит вес w и смещение b , строит линейную комбинацию и пропускает её через гиперболический тангенс:

$$z = wx + b,$$

$$\hat{y} = \tanh(z).$$

Вес меняет наклон границы между отрицательными и положительными прогнозами. Смещение перемещает границу по оси входа. Оба параметра должны быть Value, иначе механизм не построит для них градиенты.

Для одного объекта граф прямого прохода выглядит так:

Для каждого объекта создаются собственные вершины z_i , $\hat{y}_i$, e_i и q_i , но объекты weight и bias остаются одними и теми же. Поэтому обратный проход каждого объекта добавляет вклад в уже существующие weight.grad и bias.grad. Итоговое обратное направление показывает отдельная схема:

До перехода к коду проговорите порядок: функция потерь сначала делит градиент между квадратами ошибок, квадрат передаёт ошибке множитель $2e_i$, гиперболический тангенс передаёт множитель $1 - \hat{y}_i^2$, а линейная комбинация добавляет в вес множитель x_i и в смещение множитель 1.

Создаём класс TanhNeuron

В ячейке tanh_neuron сначала создайте этот каркас:

```

class TanhNeuron:
    def __init__(self, weight, bias):
        self.weight = Value(weight)
        self.bias = Value(bias)

```

Затем добавьте метод вызова нейрона и метод parameters:

```

    def __call__(self, x):
        return (self.weight * x + self.bias).tanh()

    def parameters(self):
        return [self.weight, self.bias]

```

Полная итоговая ячейка tanh_neuron должна быть такой:

Прямой вычислительный граф одного объекта нейрона

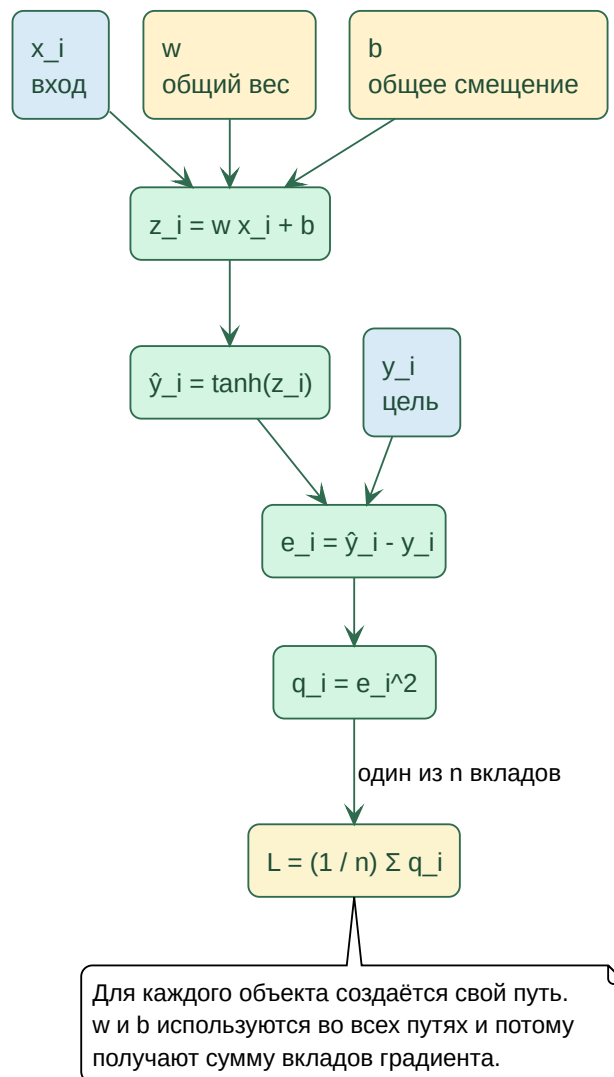


Рис. 2. Прямой граф нейрона

Обратный проход одного объекта к общим параметрам

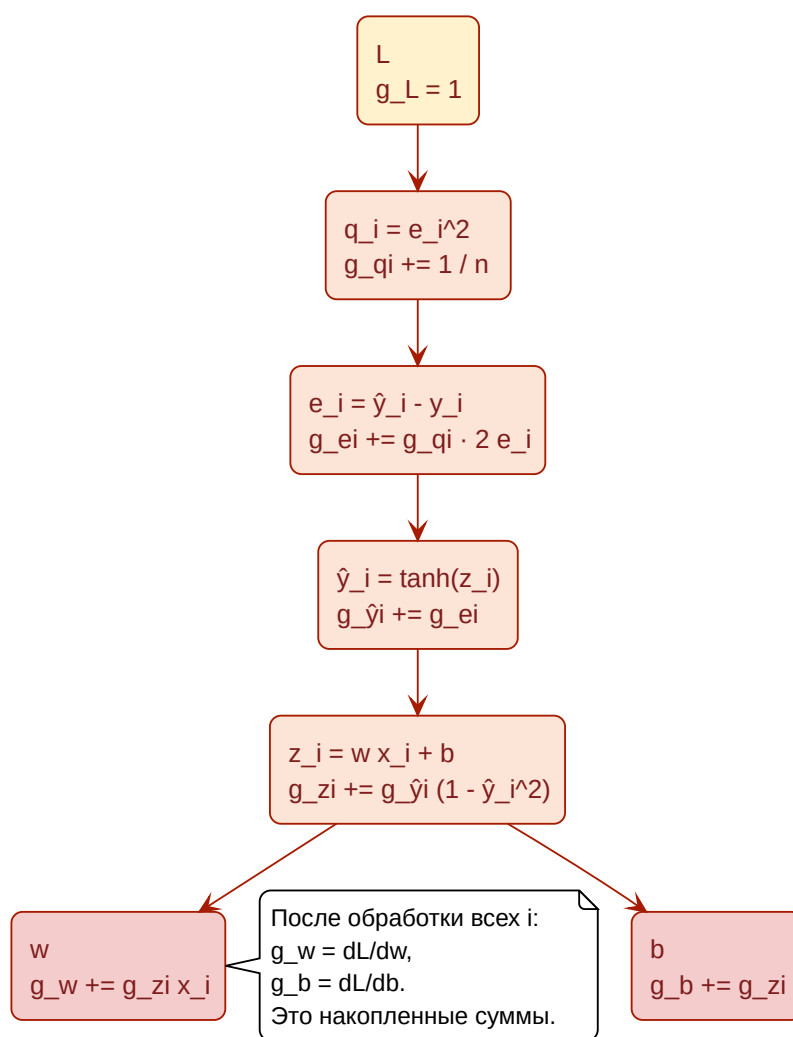


Рис. 3. Обратный граф нейрона

```
# AUTOGRAD_ESSENTIALS_CELL: tanh_neuron
class TanhNeuron:
    def __init__(self, weight, bias):
        self.weight = Value(weight)
        self.bias = Value(bias)

    def __call__(self, x):
        return (self.weight * x + self.bias).tanh()

    def parameters(self):
        return [self.weight, self.bias]
```

Объекты параметров и граф эпохи

В `__init__` создаются два долгоживущих объекта `Value`: `self.weight` и `self.bias`. Каждый новый вызов `neuron(x)` создаёт свежие промежуточные объекты умножения, сложения и `tanh`, но использует те же ссылки на `weight` и `bias`. Поэтому после построения `loss` для шести объектов у общего графа много путей к двум одним и тем же параметрам, а их поля `grad` собирают сумму вкладов всего набора.

Метод `parameters()` возвращает список ссылок на эти же объекты, а не копии их чисел. Строка `parameter.data -= learning_rate * parameter.grad` позднее меняет `data` именно тех `Value`, которые нейрон применит на следующей эпохе. После обновления строится новый `loss` и новый граф; старый граф не используют повторно для производных.

После запуска ячейки выполните ручной опыт. При $w = 2$, $b = -1$, $x = 0.5$ линейная комбинация равна 0, а $\tanh(0) = 0$.

```
# AUTOGRAD_ESSENTIALS_CELL: neuron_forward_experiment
neuron = TanhNeuron(weight=2.0, bias=-1.0)
prediction = neuron(0.5)

assert_close(prediction.data, 0.0)
assert neuron.parameters() == [neuron.weight, neuron.bias]
print("Прямой проход одного нейрона: OK")
```

Строим среднеквадратичную ошибку

Используем шесть объектов. Целевые значения слева от границы равны -1 , справа 1 :

$$x = [-3, -2, -1, 0, 1, 2],$$

$$y = [-1, -1, -1, -1, 1, 1].$$

Среднеквадратичная ошибка:

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2.$$

Вместо деления объекта `Value` на число мы умножаем сумму на обычное число $1/n$. Это сохраняет соответствие с операциями, реализованными в `Value`.

Вставьте в ячейку `loss_and_gradient`:

```
# AUTOGRAD_ESSENTIALS_CELL: loss_and_gradient
INPUTS = [-3.0, -2.0, -1.0, 0.0, 1.0, 2.0]
TARGETS = [-1.0, -1.0, -1.0, -1.0, 1.0, 1.0]

def mean_squared_loss(neuron, inputs, targets):
    total = Value(0.0)

    for input_number, target_number in zip(inputs, targets):
```

```

        prediction = neuron(input_number)
        error = prediction - target_number
        total = total + error ** 2

    return total * (1.0 / len(inputs))

def plain_loss(weight, bias, inputs, targets):
    total = 0.0

    for input_number, target_number in zip(inputs, targets):
        prediction = math.tanh(weight * input_number + bias)
        total += (prediction - target_number) ** 2

    return total / len(inputs)

gradient_neuron = TanhNeuron(weight=0.0, bias=0.0)
initial_loss = mean_squared_loss(gradient_neuron, INPUTS, TARGETS)
initial_loss.backward()

step = 1e-6
numerical_weight_gradient = (
    plain_loss(step, 0.0, INPUTS, TARGETS)
    - plain_loss(-step, 0.0, INPUTS, TARGETS)
) / (2 * step)
numerical_bias_gradient = (
    plain_loss(0.0, step, INPUTS, TARGETS)
    - plain_loss(0.0, -step, INPUTS, TARGETS)
) / (2 * step)

assert_close(initial_loss.data, 1.0)
assert_close(
    gradient_neuron.weight.grad,
    numerical_weight_gradient,
    tolerance=1e-5,
)
assert_close(
    gradient_neuron.bias.grad,
    numerical_bias_gradient,
    tolerance=1e-5,
)
print("Начальная ошибка:", initial_loss.data)
print("Градиент по весу:", gradient_neuron.weight.grad)
print("Градиент по смещению:", gradient_neuron.bias.grad)

```

При нулевых параметрах начальная ошибка равна 1. Градиент по весу отрицательный, поэтому обновление с минусом увеличит вес. Градиент по смещению положительный, поэтому обновление уменьшит смещение. Это согласуется с набором: точка $x = 0$ должна стать отрицательной.

Обучаем параметры

Каждая эпоха делает ровно четыре действия.

1. Строит новый граф и loss.
2. Вызывает loss.backward().
3. Читает градиенты weight.grad и bias.grad.
4. Изменяет data параметров по правилу градиентного спуска.

Вставьте в ячейку train_neuron полный код:

```

# AUTOGRAD_ESSENTIALS_CELL: train_neuron
learning_rate = 0.2
epochs = 80
trained_neuron = TanhNeuron(weight=0.0, bias=0.0)
losses = []

for epoch in range(epochs):
    loss = mean_squared_loss(trained_neuron, INPUTS, TARGETS)
    loss.backward()
    losses.append(loss.data)

    for parameter in trained_neuron.parameters():
        parameter.data -= learning_rate * parameter.grad

final_predictions = [
    trained_neuron(input_number).data
    for input_number in INPUTS
]
predicted_labels = [
    1.0 if prediction >= 0.0 else -1.0
    for prediction in final_predictions
]
accuracy = sum(
    prediction == target
    for prediction, target in zip(predicted_labels, TARGETS)
) / len(TARGETS)

assert losses[-1] < 0.03, losses[-1]
assert accuracy == 1.0, accuracy
assert abs(trained_neuron.weight.data) > 0.1
assert abs(trained_neuron.bias.data) > 0.1

print("Ошибка в первой эпохе:", losses[0])
print("Ошибка в последней эпохе:", losses[-1])
print("Вес после обучения:", trained_neuron.weight.data)
print("Смещение после обучения:", trained_neuron.bias.data)
print("Прогнозы:", final_predictions)
print("Точность:", accuracy)

```

На этой фиксированной задаче последняя ошибка должна быть меньше 0.03, а точность равна 1. Точные последние цифры могут немного отличаться между версиями Python, поэтому проверяется разумный порог, а не строковое совпадение вывода.

Диагностика

Ошибка: у общего узла неверный градиент

Если для $a * a + a$ получается не 5, проверьте два места.

- В локальных функциях backward должен использоваться оператор +=.
- В вершине умножения градиент self умножается на other.data, а градиент other — на self.data.

Ошибка: все градиенты равны нулю

Проверьте, что метод backward установил `self.grad = 1.0` до обхода `reversed(topo)`. Без этого в локальные правила придёт нулевой исходящий градиент.

Ошибка: появляется неверный порядок вычислений

Проверьте, что `topo` добавляет вершину после родителей, а локальные функции вызываются по `reversed(topo)`. Если идти по `topo` напрямую, родитель будет обработан раньше, чем соберёт все вклады от потомков.

Ошибка: `loss` не уменьшается

Проверьте порядок кода внутри эпохи. Сначала строится новый `loss`, затем вызывается `loss.backward()`, только потом меняются `parameter.data`. Убедитесь, что обновление использует знак `-=`, а не `+=`.

Ошибка: после второго `backward` градиент неожиданно удваивается

Учебный метод `backward` сам обнуляет `grad` всех вершин достижимого графа. Если вы меняли этот участок, верните цикл обнуления. Если вы используете старые объекты после повторного запуска ячейки класса, перезапустите эксперименты сверху вниз.

Что здесь намеренно не разбирается

Для понимания этой реализации достаточно знать ссылки на объекты, специальные методы арифметики, замыкания, список, множество и рекурсивный обход. Мы намеренно не разбираем устройство интерпретатора Python, сборку мусора, байткод и внутренний протокол вызова методов. Также эта лабораторная не описывает устройство PyTorch, многомерные тензоры, хранение промежуточных массивов и оптимизированные ядра. Эти темы важны для промышленных библиотек, но не меняют алгоритм нашего скалярного графа.

Чек-лист перед сдачей

- ☐ Я могу объяснить, что хранит каждая вершина `Value`.
- ☐ Я проверил прямой проход сложения, умножения, отрицания, вычитания, степени и элементарных функций.
- ☐ Я получил градиент 5 для $a * a + a$ при $a = 2$.
- ☐ Я сравнил градиент `Value` с центральной разностью.
- ☐ Я могу объяснить, как объекты `Value`, `parents`, замыкания `_backward`, `visited` и `+=` вместе реализуют автоматическое дифференцирование.
- ☐ Я могу записать функцию потерь нейрона формулой.
- ☐ Я получил ошибку меньше 0.03 и точность 1 после обучения.
- ☐ Я могу объяснить, почему вес и смещение стали `Value`, а не обычными `float`.

Вопросы для самопроверки

1. Почему значение a в выражении $a * a + a$ получает два вклада градиента?
2. Почему итоговая вершина получает начальный градиент 1?
3. Почему локальное правило умножения использует значение второго множителя?
4. Почему нельзя обновить вес до вызова `backward`?
5. Почему выход гиперболического тангенса подходит для целевых значений -1 и 1 ?
6. Какие возможности промышленного автоматического дифференцирования отсутствуют в этой скалярной реализации?
7. Почему `visited` добавляет объект a в `topo` один раз, но выражение $a * a$ всё равно даёт два вклада в `a.grad`?
8. Какие ссылки сохраняет замыкание `_backward` и почему обновление `parameter.data` должно происходить после `loss.backward()`?